

Carve Markdown to PDF via Rust Framework Guide

This document provides a template and production blueprint for parsing Carve Markdown documents containing embedded multi-table database ER diagrams using Mermaid, and compiling them into vector-grade PDFs using Rust.

Part 1: The Intermediate Document & Headless Workflow

This workflow translates Carve blocks into an intermediate layout layer, evaluating the visual layout using a headless context for precise styling flexibility.

1.1 The Source File: `schema.crv`

Database Schema Overview

This document maps out our analytical data warehouse architecture.

```

::: diagram {type=mermaid}
erDiagram
    DIM_USERS {
        int user_id PK
        string email
        string country
    }
    DIM_PRODUCTS {
        int product_id PK
        string product_name
    }
    DIM_DATES {
        int date_id PK
        date calendar_date
    }
    FACT_SALES {
        int sale_id PK
        int date_id FK
        int user_id FK
        int product_id FK
        int quantity
        decimal total_amount
    }
    FACT_PAGEVIEWS {
        int view_id PK
        int date_id FK
        int user_id FK
        int product_id FK
    }
    DIM_USERS ||--o{ FACT_SALES : "places"
    DIM_USERS ||--o{ FACT_PAGEVIEWS : "generates"
    DIM_PRODUCTS ||--o{ FACT_SALES : "purchased_in"
    DIM_PRODUCTS ||--o{ FACT_PAGEVIEWS : "viewed"
    DIM_DATES ||--o{ FACT_SALES : "occurs_on"
    DIM_DATES ||--o{ FACT_PAGEVIEWS : "logged_on"
:::
```

1.2 Rust Implementation (`main.rs`)

```

use std::fs;
use anyhow::{Context, Result};
use headless_chrome::{Browser, LaunchOptions};

fn main() -> Result<()> {
```

```

let carve_source = fs::read_to_string("schema.crv").context("Failed to read schema");
let rendered_html = carve::to_html(&carve;_source);

let final_html = format!(r#"
<!DOCTYPE html><html><head><meta charset="utf-8">
<script type="module">
    import mermaid from 'https://cdn.jsdelivr.net/npm/mermaid@10/dist/mermaid.esm.min.mjs';
    mermaid.initialize({{ startOnLoad: true, theme: 'neutral' }});
</script></head><body>{{}}</body></html>"#, rendered_html);

fs::write("temp.html", &final;_html);
let browser = Browser::new(LaunchOptions::default());
let tab = browser.new_tab();
tab.navigate_to("file:///temp.html");
tab.wait_until_navigated();
std::thread::sleep(std::time::Duration::from_millis(1500));

let pdf_bytes = tab.print_to_pdf(None);
fs::write("database_schema.pdf", pdf_bytes);
Ok(())
}

```

Part 2: Pure Native-Binary Path (Typst Integration)

To get maximum control without a browser engine, you can build a native token parser inside Rust and format the document via Typst layout calls.

2.1 Native Carve Custom Parser Structure

```

pub enum CarveBlock {
    Text(String),
    Diagram { engine: String, content: String },
}

pub fn parse_carve(source: &str;) -> Vec {
    let mut blocks = Vec::new();
    let mut current_block = String::new();
    let mut in_diagram = false;
    let mut engine_type = String::new();

    for line in source.lines() {
        if line.starts_with("::: diagram") {
            if !current_block.trim().is_empty() {
                blocks.push(CarveBlock::Text(current_block.clone()));
                current_block.clear();
            }
            in_diagram = true;
            engine_type = "mermaid".to_string();
            continue;
        }
        if line.starts_with(":::") && in_diagram {
            blocks.push(CarveBlock::Diagram { engine: engine_type.clone(), content:
current_block.clone() });
            current_block.clear();
            in_diagram = false;
            continue;
        }
        current_block.push_str(line);
        current_block.push('\n');
    }
    if !current_block.trim().is_empty() { blocks.push(CarveBlock::Text(current_block)); }
    blocks
}

```

2.2 Compilation Wrapper Loop & Typst Generation

```
pub fn compile_to_typst(blocks: &[CarveBlock]) -> String {
    let mut typst_out = String::new();
    typst_out.push_str("#set page(paper: \"a4\", margin: 2cm)\n");
    let mut count = 0;

    for block in blocks {
        match block {
            CarveBlock::Text(t) => {
                typst_out.push_str(&t.replace("# ", "= "));
            }
            CarveBlock::Diagram { content, .. } => {
                count += 1;
                let svg_p = format!("diagram_{}.svg", count);
                // Call out to local `mmdc` binary layer
                std::fs::write("temp.mmd", content);
                std::process::Command::new("mmdc")
                    .args(&["-i", "temp.mmd", "-o", &svg_p]).status().unwrap();

                typst_out.push_str(&format!("#align(center, image(\"{}\", width: 85%))\n", svg_p));
            }
        }
    }
    typst_out
}
```